

Anti-Bluff Masking Sweep — Client Codebase (core/, feature/)

Date: 2026-07-02 **Scope:** core/** + feature/** (Android client).
READ-ONLY audit — no source edited. **Motivating incident:** the RuTracker brotli-undecoded-body bug (`.lava-ci-evidence/sixth-law-incidents/2026-07-02-rutacker-brotli-undecoded-body.json`) — a total failure (every HTML parse got raw brotli bytes) was invisible because the broken paths degraded to "no results" / "service unavailable" instead of surfacing an error, and login (header-only) appeared to succeed. **Vocabulary:** per §11.4.6 — statements are facts backed by cited `file:line` evidence; inferences that were not executed are marked UNCONFIRMED:.

Bug class definition

A **masking bug** here = a real, total failure (stale selector, undecoded body, upstream-shape change, parser regression, hard down) that the code converts into a benign-looking outcome — an empty list ("no results"), a null, a false, or a *transient*-flavoured error state ("Service unavailable, try again later") — so that: (a) no test fails, and (b) the user cannot tell "broken" from "legitimately empty / temporarily down".

Ranked findings

#	Sev	File:line	Mas
F1	P0	core/tracker/rutacker/.../impl/ RuTrackerNetworkApi.kt:51-91 + .../feature/ RuTrackerAuth.kt:60-70	catch Auth map stale pars Serv

#	Sev	File:line	Mas
F2	P0	core/tracker/rutracker/.../domain/ GetSearchPageUseCase.kt:56; core/tracker/kinozal/.../ parser/KinozalSearchParser.kt:31-33; core/tracker/ nnmclub/.../parser/NnmclubSearchParser.kt:33-35; core/ tracker/rutor/.../parser/RuTorSearchParser.kt:52-57	Search select what misr login body
F3	P1	app/src/androidTest/.../ Challenge02AuthenticatedSearchOnRuTrackerTest.kt:170-177 and :185-195	In-bo succ (Com that unav not l
F4	P1	core/tracker/rutracker/ (whole module)	Zero (find '*fi kino does
F5	P1	core/tracker/kinozal/.../http/KinozalHttpClient.kt:203	body retu body
F6	P2	core/tracker/client/.../LavaTrackerSdk.kt:276-283 (getTopic), :300-307 (downloadTorrent), :324-336 + :343-350 (downloadHttpFile)	catch torre even
F7	P2	core/tracker/rutor/.../feature/RuTorTopic.kt:73-90	fetch empty

#	Sev	File:line	Mas
			degr frag
F8	P2	core/tracker/kinozal + core/tracker/nmclub fixtures	New ...-20 capt ...-20
F9	P3	app/src/androidTest/.../Challenge00...:160, Challenge02...:118, Challenge03...:100, Challenge12...:77, Challenge20...:83, Challenge23...:94	try catch selec
F10	P3 (not-a-bug, recorded)	KinozalClient.kt:40-49, ArchiveOrgClient.kt:47-53, RuTrackerAuth.kt:85-89, all tracker healthCheck()	catch

Item-by-item against the task's four asks

1. bodyAsText() / .body after a manual content-encoding header — beyond the 2 fixed rutracker paths

FACT: no other instances exist. A repo-wide scan (`grep -rn "Accept-Encoding\|acceptEncoding\|Content-Encoding" over core/feature/ --include=*.kt`) matches **only** the two rutracker files, and both now correctly **omit** the header:

- `core/tracker/client/.../di/TrackerClientModule.kt:94-108` — comment + no header (fixed).
- `core/tracker/rutracker/.../RuTrackerHttpClientFactory.kt:56-64` — comment + no header (fixed, clone path).

All other HTTP clients are raw `OkHttp` (`RuTorHttpClient`, `NmclubHttpClient`, `KinozalHttpClient`, `ArchiveOrgHttpClient`, `GutenbergHttpClient`, `IPTorrents`) or `Ktor-on-OkHttp`

(NetworkApiRepositoryImpl lava-api-go client). **None set a manual Accept-Encoding**, so OkHttpClient adds its own Accept-Encoding: gzip and transparently decompresses — the brotli failure mode cannot occur there. GutenbergHttpClient.kt:38 sets only Accept: application/json. (The "br" and node.tag().name == "br" hits are the HTML
 tag in ParsePostUseCase.kt:109, unrelated to brotli.)

2. catch/runCatching/getOrNull/?: emptyList()/?: return that maps a real failure to a generic benign result

Ranked above. The load-bearing ones are **F1** (login Throwable→ServiceUnavailable = the described class, verbatim), **F2** (search parse-mismatch→emptyList, systemic across 4 trackers), **F6** (SDK topic/download→null). The lava-api-go client path is **clean** — ApiBackedTrackerClient.getString/getBytes/postJson (:270-319) throw ApiHttpException with the real status code on any non-2xx, and ProbeMirrorUseCase (core/domain/.../ProbeMirrorUseCase.kt:17-53) returns a **distinguishable** Unhealthy(status) vs Unreachable(reason) — neither masks. ProviderCatalogRepository.fetchProviders (core/data/.../ProviderCatalogRepository.kt:100-122) returns Result<> with the failure reason — not masking.

3. Stale HTML selectors — parsers that throw vs return-empty on mismatch

- **Throws (good, surfaces the problem):** GetCurrentProfileUseCase.parseUserId (:30) error("... selectors stale"); LoginUseCase (:40,43) throws Unknown/NoData. **BUT** every one of these throws is caught by F1's login catch and flattened to ServiceUnavailable, so the honest throw is re-masked downstream.
- **Returns empty (masks, = F2):** all four HTML **search** parsers (rutracker/kinozal/nnmclub/rutor) return emptyList() when the row/container selector matches nothing. None assert the results container is present.
- **Partial-throw hazard:** GetSearchPageUseCase.parseSearchPage:67 does element.select(".tor-size").attr("data-ts_text").toLong() — if .hl-tr matches but the size cell selector is stale, "".toLong() throws NumberFormatException (surfaced). Only the **total** no-.hl-tr case is silently empty. So rutracker search fails *loud* on partial drift, *silent* on total drift.
- **Fixture freshness:** rutracker = **0 fixtures** (F4). kinozal/nnmclub = 2026-05-02 (F8, ~2mo). rutor = 2026-07-02 (fresh).

4. Challenge/tests that PASS or SKIP while the feature is broken — beyond Challenge70

- **F3 — Challenge02AuthenticatedSearchOnRuTrackerTest** is the primary green-on-skip: `assumeTrue(false)` on the `ComposeTimeout / ServiceUnavailable` branches converts a totally-broken `rutracker login+search` into SKIP.
 - UNCONFIRMED: Challenge09 (Kinozal) :70,74 and Challenge10 (NNM) :58,62 use `assumeTrue` for missing credentials — this is the *legitimate* top-of-test skip pattern, but they should be re-read to confirm they do not also `assumeTrue` on a post-action failure the way C02 does.
 - F9 — swallowed `AssertionError` in provider-selection helper clicks (green-on-swallow if any wraps a primary step).
-

Top-3 (for the caller)

1. **F1 (P0)** — `RuTrackerNetworkApi.kt:51-91 + RuTrackerAuth.kt:60-70`: login maps **every** Throwable (including stale-selector `error()` and undecoded-body `NoData`) to the *transient* `ServiceUnavailable`, so a permanently-broken login shows "Service unavailable, try again later" forever. This is the described bug class verbatim; the honest throws in `LoginUseCase/GetCurrentProfileUseCase` are re-masked here. Fix: classify parser-broken vs transport-down, `recordNonFatal` the former.
2. **F2 (P0)** — the four HTML search parsers (`GetSearchPageUseCase.kt:56`, `KinozalSearchParser.kt:31`, `NmclubSearchParser.kt:33`, `RuTorSearchParser.kt:52`) return `emptyList()` on any structural mismatch; `LavaTrackerSdk.kt:886-896` then emits `ProviderResults(empty)` (not `ProviderFailure`), so a dead/blocked/undecoded provider renders as "No results found". Fix: throw `UnrecognizedPageException` when the results **container** is absent.
3. **F3 (P1)** — `Challenge02AuthenticatedSearchOnRuTrackerTest.kt:170,185`: in-body `assumeTrue(false)` on `timeout/ServiceUnavailable` turns a totally-broken `RuTracker` flow into SKIP (counts green) — the one Challenge that could catch F1/F2 for `rutracker` cannot fail on the failure it exists to detect. Fix: `fail()` when the action executed but produced neither expected outcome.

Cross-cutting root cause: the client treats "empty result set" and "null" and "ServiceUnavailable" as terminal *success-shaped* states with no separation between "legitimately empty / temporarily down" and "structurally broken". The durable fix is a **typed `UnrecognizedPage/ParserBroken` signal** at every scrape

boundary, routed to `ProviderFailure/Error+Retry` and `recordNonFatal ($6.AC)`, so total breakage is always loud — in the UI and in Crashlytics — instead of degrading to "nothing found".